

Programación orientada a objetos (introducción)

1º CFGS Desarrollo de Aplicaciones Web

Curso 2023/2024

Profesor: José Miguel Blázquez

Departamento de Informática

Introducción al concepto

- Nueva forma de pensar y diseñar software.
- Evolución de la programación modular: cambios en la forma de descomponer problemas...
- Programación imperativa
 - Programación estructurada
 - Programación modular
 - Programación **orientada a objetos**
- Intento de dominar la complejidad del software imitando la realidad.
- Es el estilo más utilizado en la actualidad.

POO - lenguajes

- La programación orientada a objetos es un **estilo** de programación, enfocado en el concepto de “**objeto**”.
- **Java** es POO pero se considera un lenguaje híbrido.
- Primero aprenderemos la “**filosofía**” y después el **lenguaje ...**
 - porque “filosofía” sólo hay una y lenguajes orientados a objetos hay muchos.
- Evolución:
 - Smalltalk 1er lenguaje POO.
 - Con C++ se consolida hacia finales de los 80.
 - Las GUI consolidan el paradigma hasta la actualidad.

Objetos

- Entidades que representan un **concepto**.
 - Ejemplo: personas.
 - No necesariamente material, puede ser intangible.
- Contienen 2 partes diferenciadas:
 - 1) **Datos**: describen sus características (variables).
 - 2) **Operaciones**: que pueden realizar (métodos o “funciones”).
- En POO son elementos que poseen:
 - **Estado**: valores de sus atributos (pelo = rubio, edad = 20).
 - **Comportamiento**: métodos definidos para realizar acciones (andar, saltar, correr).
 - **Identidad**: forma de identificar cada objeto (DNI).

POO: un antes y un después

- Hasta ahora: PE/PM descompone en **acciones**.
 - Las funciones y procedimientos eran independientes y pertenecían al programa en general.
- A partir de ahora: POO descompone en **objetos**.
 - Los objetos y clases serán propietarias de las funciones (métodos).
 - Gozarán de cierta independencia y serán coordinadas por un programa principal (también un objeto).

Características de la POO

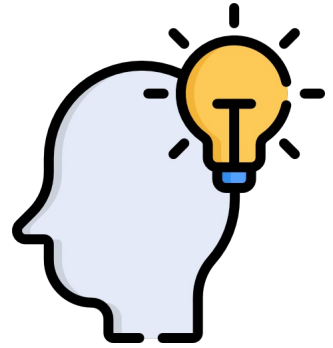
Básicas

- **Abstracción:** captar las características y funcionalidades que un objeto desempeña, y que le permite distinguirse de los demás, descartando todo aquello que no sea relevante.
- **Encapsulamiento:** reunir todos los elementos que pertenezcan a una misma entidad.
- **Ocultación (público / privado):** proteger el contenido encapsulado evitando su acceso directo desde el exterior.

- Herencia.
- Polimorfismo.

Avanzadas

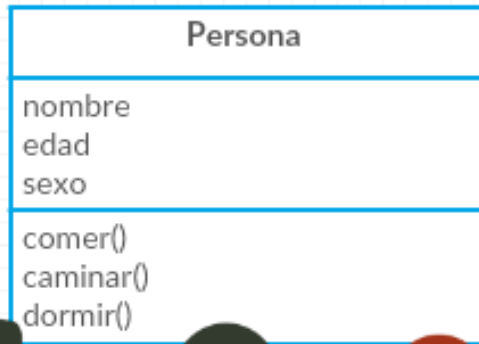
Beneficios de la POO



- **Reutilización** del código.
- **Simplifica** la complejidad de la realidad.
- Evita la duplicidad de código y mejora su legibilidad.
- Facilita **trabajar en equipo** gracias al encapsulamiento.
- Mejora trazabilidad y corrección de errores.
- Protege la información.

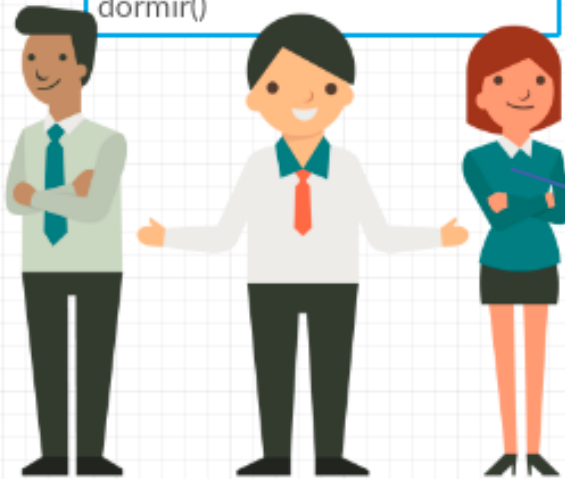
Clases y objetos

Los objetos se abstraen en clases.



una clase
(plantilla)

```
class Persona{  
    String nombre;  
    int edad;  
    String sexo;  
  
    void comer();  
    void caminar();  
    void dormir();  
}
```



nombre = "María";
edad = 30;
sexo = "Mujer";

muchos objetos (instancias de la clase Persona)

Ejercicio

- Aplicación para gestión básica de un taller de reparación de coches.
- En grupos de 3 vais a:
 - Identificar objetos necesarios en la aplicación
 - Identificar características y funciones de cada objeto



Clases: sintaxis y estructura básica

```
class Persona{  
    String nombre;  
    String apellidos;  
    String telefono;  
  
    String getNombre(){  
        return nombre;  
    }  
  
    String getApellidos(){  
        return apellidos;  
    }  
  
    void saludar(){  
        System.out.println("Hola");  
    }  
}
```

Atributos

Métodos

En esta sintaxis falta indicar la visibilidad (public, private, protected) de cada elemento, pero por simplicidad no aparece.

Acceso a miembros de una clase

- Los atributos y métodos pueden definirse con estas visibilidades:
 - **public** (para los métodos de la interfaz)
 - **private** (para los atributos)
 - **protected** (para atributos en herencia)
- Consecuencias: ¿cómo accedemos a los atributos?
 - Setters
 - Getters

Visibilidad miembros y acceso

```
class Persona{  
    private String nombre;  
    private int edad;
```

```
    public String getNombre(){  
        return nombre;  
    }
```

```
    public int getEdad(){  
        return edad;  
    }
```

```
    public void setNombre(String nom){  
        nombre = nom;
```

```
    public void setEdad(int age){  
        edad = age;  
    }
```

```
}
```

getters

setters

Programa principal

```
public static void main(){
```

```
    Persona p1;  
    p1 = new Persona();
```

```
    p1.setNombre("Paco");  
    p1.setEdad(45);
```

```
    p1.edad = 45;
```

```
    int edad = p1.getEdad();  
    edad++;
```

```
    System.out.print(edad);
```

```
}
```

¿Salida en
pantalla?

Podríamos hacer esto si los atributos no fueran "private". PERO esta práctica no debe hacerse bajo ningún concepto. Se rompe la ocultación.

Setters y getters

- ¿Porqué usarlos y no acceder directamente a los atributos? Encapsulamiento y ocultación.
- Entre muchas cosas que requieren conocimientos más avanzados en POO, destacan:
 - Implementar validaciones y/o restricciones, especialmente en los setters.
 - Proteger algunos campos del acceso directo de usuarios, campos que no deben ser consultados (no creo el getter) o campos de sólo lectura (no creo el setter).
 - Los getters y setters sólo se definirán cuando se necesite que un atributo tenga comunicación con el mundo exterior.
 - La aplicación de la herencia y el polimorfismo requieren la existencia de estos métodos: veremos ejemplos de porqué más adelante...

Setters y getters

```
public class Persona {  
    private String nombre;  
  
    public String getNombre() {  
        if(nombre.length()>16)  
            return "¡El nombre es muy largo!";  
        else  
            return nombre;  
    }  
  
    public void setNombre(String name) {  
        if(name!=null && name.length()>3)  
            nombre = name;  
    }  
}
```

Limitador velocidad
máxima en coches
gama alta

```
public void setSpeed(int newSpeed)  
{  
    if (newSpeed > 250) {  
        speed = 250;  
    } else {  
        speed = newSpeed;  
    }  
}
```

Constructores

- Método inicialización creación instancia.
 - También se denomina: instanciar
- Tipos:
 - Constructor por defecto (Java genera uno...)
 - Constructores definidos (y sus variaciones)

```
class Persona{  
    private String nombre;  
    private int edad;  
  
    public Persona(String name, int age){  
        nombre = name;  
        edad = age;  
    }  
}
```

```
Persona p1;  
p1 = new Persona("Jose", 27);
```

Llamada al constructor

¿Porqué "name" y no "nombre"?...

Constructores

```
class Persona{  
    private String nombre;  
    private int edad;  
  
    public Persona(String name, int age){  
        nombre = name;  
        edad = age;  
    }  
  
    public Persona(){  
        nombre = "Anónimo";  
        edad = 0;  
    }  
}
```

2 constructores diferentes...

Al crear el objeto llamaremos a uno o a otro...

Los **destructores** son métodos que se encargan de eliminar los objetos de memoria cuando ya no son necesarios...en Java no existen. Su trabajo lo realiza automáticamente el “Recolector de basura” sin intervención del programador...otros lenguajes como C++ sí lo requieren.

Palabra reservada “this”

```
12 public class Artículo {
13     // Atributos de la clase
14     private String cod;
15     private String titulo;
16     private String formato;
17     private float precio_alquiler;
18
19     // Constructor de la clase
20     public Artículo(String cod, String titulo, String formato, float precio_alquiler)
21     {
22         // Con 'this' estamos accediendo al objeto de la clase
23         this.cod = cod;
24         this.titulo = titulo;
25         this.formato = formato;
26         this.precio_alquiler = precio_alquiler;
27     }
28     // Métodos de la clase
29     public float precio1(){
30         return (getPrecio_alquiler());
31     }
}
```

“this” permite eliminar ambigüedad cuando usamos el mismo nombre para 2 variables del mismo ámbito siendo una de ellas del propio objeto

Constantes de clase y objeto

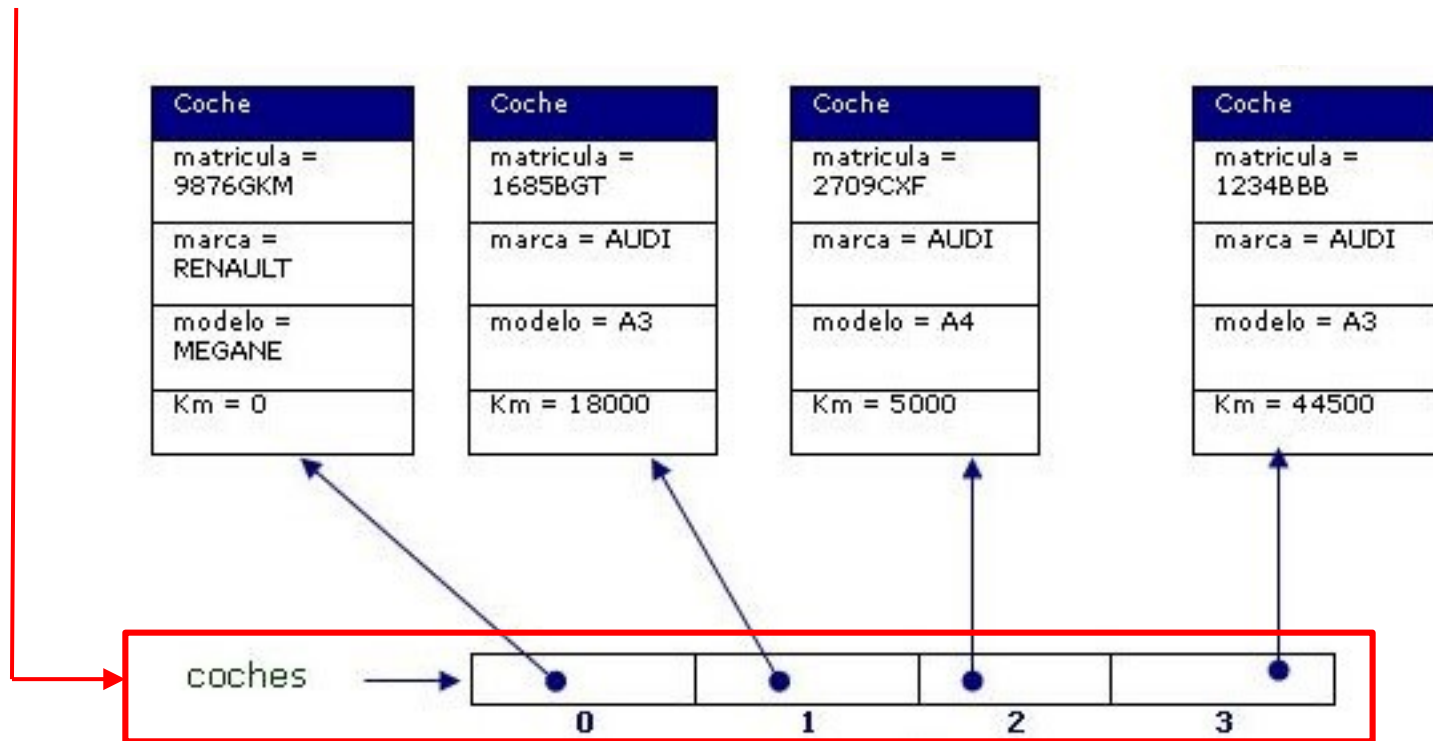
- final: definir constantes, pueden ser diferentes para cada instancia. Los inicializa el constructor y ya no varían.
 - gravedad de un planeta.
- static: atributos con valores comunes a todos los objetos. *Pueden usarse sin instanciar la clase.* Similar a variables globales.
 - Métodos estáticos (visto en tema anterior): llamar sin instanciar (`Math.sqrt(9)`)
 - En variables debe limitarse su uso. Dificultan depuración de errores y van contra el encapsulamiento.

Ejercicio

- Crea una clase llamada Persona. Sus atributos son: nombre, apellidos, edad, sexo, peso, altura y DNI. Construye los siguientes métodos para la clase:
 - Setters y getters para cada uno de los atributos. Hay que **validar** las entradas de datos de valores obvios.
 - Mostrar en pantalla los datos de la persona de forma “agradable”.
 - Devolver un valor booleano indicando si es o no mayor de edad.
 - Saludar diciendo su nombre.
 - Generar DNI aleatorio (válido).
 - Calcular el IMC: indica si la persona esta en su peso ideal (peso en kg/(altura² en m)), si el cálculo resulta en valor < 20, la función devuelve un -1; si el cálculo está entre 20 y 25 (incluidos), significa que está por debajo de su peso ideal y la función devuelve un 0; y si el cálculo es un valor > 25 significa que tiene *sobrepeso*, la función devuelve un 1.
- Realiza un programa principal que cree 4 personas y te pida sus datos (salvo el DNI que debe generarse). Tras esto, muestra en pantalla los datos de cada una, que saluden e indica su IMC de forma cualitativa.

Arrays de objetos

- Pueden ser estáticos o dinámicos...
- `Coche[] coches = new Coche[4];`

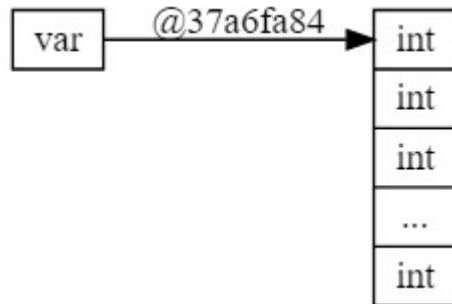


```
coches[0] = new Coche("9876GKM","RENAULT","MEGANE",0);
```

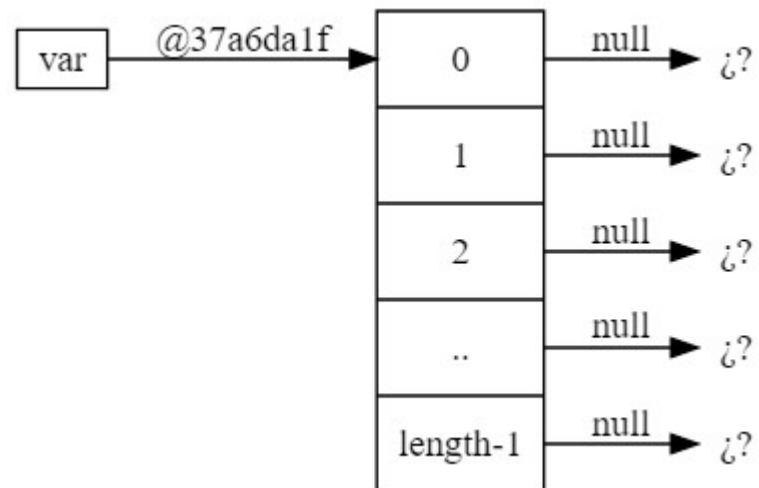
Arrays de tipos simples vs objetos

```
TipoDato[] var = new TipoDato[n];
```

Si *TipoDato* es un tipo de dato **simple**, la instrucción *new* reserva la cantidad **exacta** de memoria.



Pero si *TipoDato* es un **objeto**, un *new* sólo creará **referencias** vacías a futuros objetos que, cuando sean creados, podrán almacenarse.



Arrays de objetos II

```
class Estrella {  
    float x;  
    float y;  
}
```

```
Estrella[] listaEstrellas = new Estrella[10];
```

Esto no crea una Estrella, sino el “recipiente” para almacenar una Estrella en el Array.

```
for (int i=0; i<listaEstrellas.length; i++) {  
    listaEstrellas[i] = new Estrella();  
    listaEstrellas[i].x = random(0,width);  
    listaEstrellas[i].y = random(0,height);  
}
```

Esto sí crea una Estrella.

```
for (int i=0; i<listaEstrellas.length; i++) {  
    System.out.println(listaEstrellas[i].x, listaEstrellas[i].y);  
}
```

Esto es posible porque NO son privados, pero deberían serlo... ¿cómo cambiarían las cosas entonces?